# Solution: Intersection of Two Arrays

## Problem Statement

Given two arrays, `nums1` and `nums2` containing integers, return an array of the intersection of `nums1` and `nums2`. In the resultant array, each integer should be unique, and elements can be in any order.

### Examples

- **Example 1**:
  - Input: `nums1 = [3,1,2,6,7,8]`, `nums2 = [2,3,4,5,7,9]`
  - Expected Output: `[2,3,7]`
  - Justification: The numbers 2, 3, and 7 are present in both `nums1` and `nums2`.
- **Example 2**:
  - Input: `nums1 = [1,2,3]`, `nums2 = [4,5,6]`
  - Expected Output: `[]`
  - Justification: There are no common elements between `nums1` and `nums2`, hence the output is an empty array.
- **Example 3**:
  - Input: `nums1 = [7,8,9,10]`, `nums2 = [10,9,8,7]`
  - Expected Output: `[7,8,9,10]`
  - Justification: All elements from `nums1` are present in `nums2` and vice versa.

## Solution

To solve this problem, we will utilize a set-based approach. Sets are ideal for this task because they automatically remove duplicate elements and allow for efficient element lookup. First, we'll convert both input arrays into sets. This conversion eliminates any duplicates within each array, aligning with our requirement to return only unique common elements. Then, we'll find the intersection of these two sets, which gives us the set of elements present in both arrays. This method is effective because set operations are generally more efficient than iterating through arrays, especially when dealing with duplicates and the need for uniqueness. Our approach reduces the problem to simple set operations, making the solution more elegant and performant.

The rationale behind using sets over other data structures is twofold. Firstly, the automatic removal of duplicates simplifies our task. Secondly, the intersection operation directly corresponds to our goal of finding common elements, ensuring that the algorithm is both intuitive and efficient.

### Step-by-step Algorithm

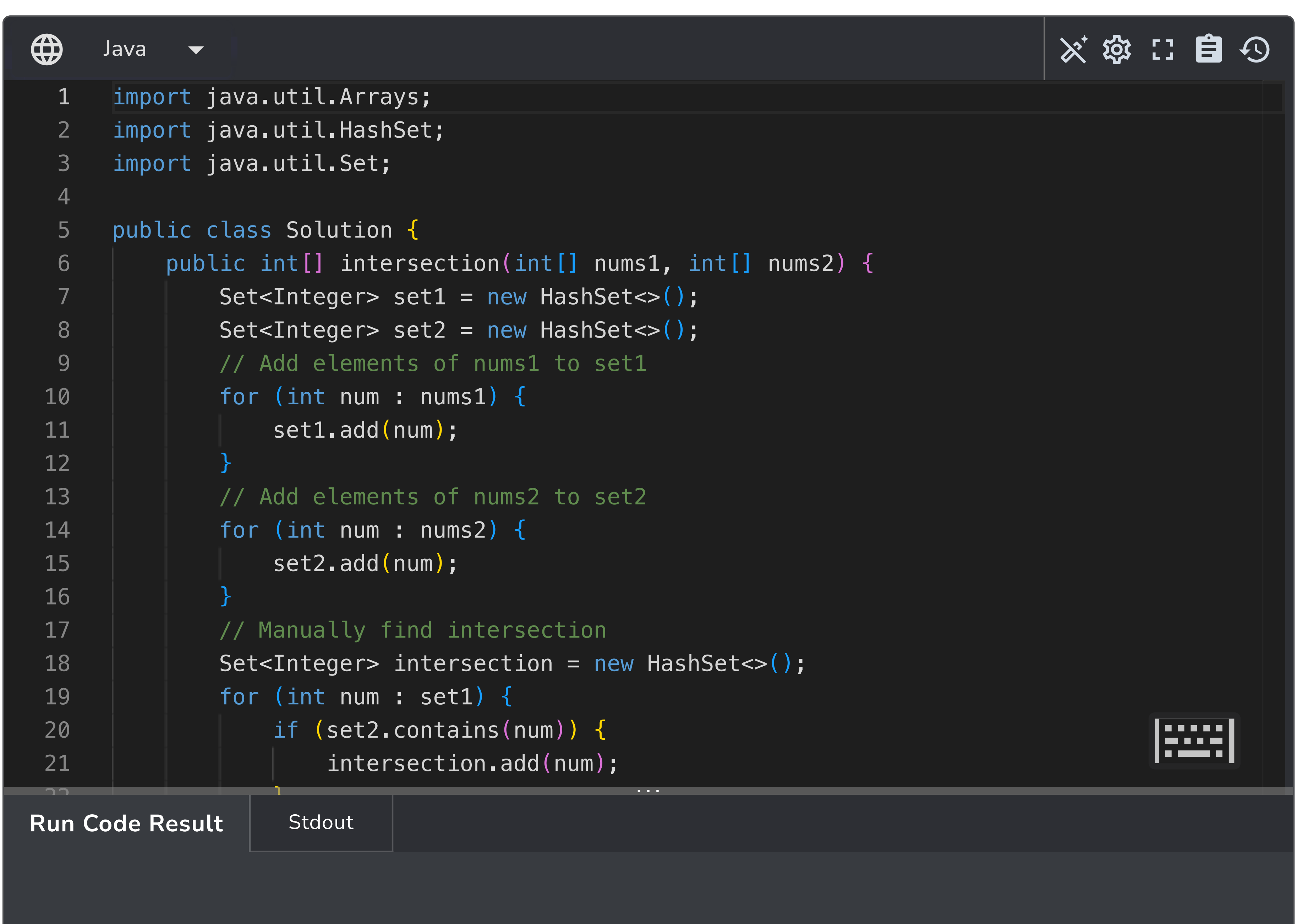
1. **Initialize Two Sets**: Start by creating two empty sets, `set1` and `set2`. These will store the unique elements from `nums1` and `nums2`, respectively.
2. **Populate the First Set** (`set1`): Loop through each element in `nums1`. For every element, add it to `set1`. This step ensures that `set1` contains all unique elements from `nums1`.
3. **Populate the Second Set** (`set2`): Repeat the process for `nums2`, adding each of its elements to `set2`. Now, `set2` contains all unique elements from `nums2`.
4. **Find Intersection**: Identify the common elements between `set1` and `set2`. This can be done by checking each element of `set1` to see if it also exists in `set2`, or by utilizing set intersection operations available in many programming languages.
5. **Prepare the Result**: Convert the intersection set into an array. This step involves initializing an array and populating it with the elements from the intersection set.
6. **Return the Result**: Finally, return the array created in the previous step. This array contains all unique elements that are present in both `nums1` and `nums2`.

### Algorithm Walkthrough with Input

Given `nums1 = [3,1,2,6,7,8]`, `nums2 = [2,3,4,5,7,9]`:

1. **Initialize Sets**:
   - `set1 = {}`
   - `set2 = {}`
2. **Populate** `set1`:
   - Loop through `nums1`: add 3, 1, 2, 6, 7, and 8 to `set1`.
   - After the loop, `set1 = {1, 2, 3, 6, 7, 8}`.
3. **Populate** `set2`:
   - Loop through `nums2`: add 2, 3, 4, 5, 7, and 9 to `set2`.
   - After the loop, `set2 = {2, 3, 4, 5, 7, 9}`.
4. **Find Intersection**:
   - Identify common elements between `set1` and `set2`.
   - Intersection: `{2, 3, 7}`.
5. **Prepare the Result**:
   - Convert the intersection set to an array: `[2, 3, 7]`.
6. **Return the Result**:
   - The final result is `[2, 3, 7]`, which is the array of unique elements present in both `nums1` and `nums2`.

## Code



```java
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class Solution {
    public int[] intersection(int[] nums1, int[] nums2) {
        Set<Integer> set1 = new HashSet<>();
        Set<Integer> set2 = new HashSet<>();
        // Add elements of nums1 to set1
        for (int num : nums1) {
            set1.add(num);
        }
        // Add elements of nums2 to set2
        for (int num : nums2) {
            set2.add(num);
        }
        // Manually find intersection
        Set<Integer> intersection = new HashSet<>();
        for (int num : set1) {
            if (set2.contains(num)) {
                intersection.add(num);
```

You must run your code first

## Complexity Analysis

### Time Complexity: $O(n+m)$

- **Creating Sets:** $O(n)$ for the first array and $O(m)$ for the second, where `n` and `m` are the lengths of the two input arrays.
- **Finding Intersection:** Iterating through the smaller set and checking for each element in the larger set, leading to $O(min(n,m))$ in the best case and $O(n+m)$ in the worst case.

Thus, the overall time complexity combines these steps, resulting in $O(n+m)$.

### Space Complexity: $O(n+m)$

- Space is required for storing elements of both arrays in sets, leading to a space complexity of $O(n+m)$ in the worst case when all elements are unique.

← Previous
Intersection Of Two Arrays (Easy)

Next →
Find The Index Of The First Occurrence In A String (Easy)

☐ Mark as Completed